

Two Models, One Compiler

A coordination layer for Qwen 3.5 Flash + GPT-OSS-120B proving competition math in Lean 4

Verified Mechanisms take-home — submission writeup. Longer-form appendices live in this repository: [RESEARCH.md](#) (design analysis), [PART2.md](#) (full Part-Two answer), [EXPERIMENTS.md](#) (chronological log of every keyed run), [RESEARCH_LOOP.md](#) (variant selection), [RUNBOOK.md](#) (reproduction). Developed with substantial assistance from Claude (Anthropic), disclosed in the submission form and in [README.md](#).

1. Problem statement

Two fixed models — [qwen/qwen3.5-flash-02-23](#) (fast, cheap, 2026 training cutoff) and [openai/gpt-oss-120b](#) (slow on our price tier, cheaper still, strong at high reasoning effort, June-2024 cutoff) — must jointly solve competition math problems and prove the answers in Lean 4 against a pinned Mathlib. A problem scores one point only if the official Comparator accepts every required declaration at the kernel level, numeric answers are decimal literals, spend stays under **\$1.00**, and the problem finishes within **8 hours**. Part One asks for the coordination layer; Part Two asks whether the collaboration actually beats either model alone, and where one model fills the other's gaps.

Our one-sentence thesis, formed early and stress-tested for a week: **this task is a coverage problem attached to a perfect verifier, and almost everything that matters follows from taking the verifier seriously** — including which "collaboration" mechanisms are worth building (three narrow ones) and which published multi-agent patterns to reject (most of them).

2. What the harness itself dictates

Before any literature, we read the kit's code, because the scoring path constrains the design more than any paper (details: [RESEARCH.md](#) §1).

The REPL you iterate against is not the gate you are scored by. The development REPL strips imports, runs in a warm full-Mathlib environment, and never compares statements; the Comparator builds a real file cold and demands kernel-level statement equality with a three-axiom whitelist. Three consequences shaped the agent: models never touch theorem statements (we splice only proof bodies and helper lemmas into the pristine challenge, so statement equality holds *by construction*); `sorry`, `native_decide`, and stray axioms are lexically banned and audited with a live `#print axioms` gate; and — the deepest lesson, found only in week two — a proof can be REPL-accepted yet *time out* the Comparator's cold build, so the agent ultimately learned to consult the real gate before trusting a win (§4.3).

The budget ledger is fail-closed. Any unaccounted transport failure permanently closes a problem's ledger: every later call raises, and the final status is worth zero even if a correct proof is checkpointed. (We adopted the upstream kit fix verbatim when it merged: HTTP 429 refusals that provably cost nothing release the reservation; everything else stays fatal.) This mechanic, not taste, dictates the agent's rails: self-imposed deadlines with margin so no call outlives the window, generous explicit timeouts, checkpoint-every-improvement, and a degraded LLM-free mode that still finalizes the best candidate after a ledger death.

Money is not the binding resource. Observed per-problem spends run \$0.01–0.10 against the \$1.00 cap even on eight-hour runs (worst case all week: \$0.95). A dollar buys ~3.8M qwen output tokens. What actually binds is **capability × wall-clock**: gpt-oss arrives on the cheap/slow provider tier (2–8 min per high-effort call), and serial REPL checks are the other clock. Every architectural decision allocates time, not dollars.

The committed baselines are a controlled experiment someone already ran for us. The kit ships two single-model repair-loop baseline runs. Their pathologies are strikingly complementary: gpt-oss is *stable-but-stuck* (on one problem it produced 25 consecutive turns with the byte-identical error, having had the right numeric answer from turn 1), while qwen is *diverse-but-thrashing* (25 turns, a different error signature nearly every turn, hallucinated lemma names). qwen solved 7/16, gpt-oss 8/16 — **union 10/16**. Two more facts fell out of the pooled 3,900-check error taxonomy: "no goals" proof-script misalignment dominates (2,530 occurrences -> structured `have`-chains and goal-state-in-prompt repair), and hallucinated constants are second (622 -> name verification before burning repair rounds).

3. Prior work, and the architectures we did not build

The 2025–26 literature on LLM theorem proving and multi-agent coordination is large; what mattered was filtering it through one question: *what still pays when an exact verifier judges every candidate for free?* Full review with citations in [RESEARCH.md §4](#); the arXiv references below are representative.

Evidence we adopted. (1) *Repeated sampling against a verifier* converts coverage directly into accuracy (Large Language Monkeys, 2407.21787) — the null hypothesis every fancier mechanism must beat at equal cost. (2) *Cross-family union coverage*: pooling candidates from different model families yields most of the ensemble benefit with just two models, and pays exactly when the selector is strong (2510.21513, 2603.20324); ours is perfect, and the kit's own baselines showed 7/8->10. (3) *Compiler-feedback repair, kept shallow*: one or two error-informed rounds are worth roughly an order of magnitude of extra samples (Kimina; Goedel-Prover-V2; APOLLO 2505.05758; Delta Prover 2507.15225 ablation), but repair plateaus — so after 2–3 rounds, re-diversify or restructure rather than grind. (4) *Cross-model handoff with accumulated error history*: models are bad at locating their own errors but good at fixing located ones (2311.08516), and stronger/different feedback at a fixed budget beats self-repair (2306.09896) — the compiler locates for free; the second model's value is different priors. (5) *Decomposition for the hard tail*: sketch -> sorry-scaffold -> fill is the dominant architecture of every recent strong system (DSP 2210.12283 -> DSP+ -> DeepSeek-Prover-V2 2504.21801 -> Hilbert 2509.22819 -> LEAP), and it is exactly what the kit's six baseline-unsolved problems need. (6) *Verifier-gated cascade*: cheap first, escalate on failure (FrugalGPT 2305.05176) — the deferral signal is the compiler. (7) *Self-consistency only where the verifier is blind*: committing to a numeric answer before spending proof effort (2203.11171), treating disagreement as "escalate," not "outvote."

Evidence we rejected — this is a design choice, not an omission. Multi-agent *debate* repeatedly fails compute-matched replication, and more rounds can hurt (2310.01798, 2311.17371, 2502.08788, 2502.19130). *Mixture-of-Agents synthesis* loses to self-consistency variants and won 0/42 tasks in a careful head-to-head (2502.00674, 2603.20324); union + exact selection captures the value without an aggregator. *Free-form critic agents* are redundant next to a compiler that critiques perfectly. *Learned routers and bandit allocation* need training data we don't have and compete with "stop on success" plus a fixed ladder. *Tactic-level tree search* (COPRA-style) fights this kit's interface: the REPL is file-level and serial. A two-model conversation protocol was therefore never built; the models never talk to each other. They share **artifacts** — candidate files, error histories, skeletons — through three narrow interfaces, with the compiler between them. The strongest recent systems point the same way: simple loops around a strong verifier beat elaborate orchestration (Delta Prover's plain agent loop is SOTA over fine-tuned provers).

4. Our architecture

Design principle: one anytime loop, three narrow cross-model interfaces, every mechanism traceable to a failure visible in data we possess. The agent ([submission/agent.py](#) + [submission/lean_text.py](#), ~1,900 lines over the kit's services) is a staged escalation ladder; each stage is individually switchable, which is also what made Part Two's ablations honest.

4.1 The ladder

- **Rails (always on).** Self-deadline at `time_limit - verify_reserve - margin`; no LLM call may outlive it. Checkpoint every improvement. On ledger death, continue in LLM-free mode and finalize the best candidate. Durable per-problem state ([agent_state.json](#)) so an interrupted run resumes mid-plan with its lemma pool and best skeleton intact.
- **S0 — deterministic sweep (\$0).** ~15 statement-spliced tactic templates (`norm_num`, `nlinarith` with generic hint patterns, `decide` with the right `set_options`, ...). Solves the easy third of the kit set for free — every LLM dollar goes to problems that need it. Three cascade entries were later *mined offline* against archived failed subgoals from real runs.
- **S0.5 — answer consensus (answer problems only).** The one place model *agreement* is used, because it is the one decision the verifier can't check cheaply: both models independently propose the numeric answer (closed forms evaluated to decimal literals via `sympy`); agree -> commit; disagree -> one adjudication round plus REPL falsification probes (`decide` on small instances). A wrong committed answer means unbounded wasted proof search — and the baselines showed gpt-oss burning 24 turns on a proof whose answer it had in turn 1.

- **S1 — pooled diverse sampling.** A wave of cheap qwen-fast whole-file candidates plus thinking-qwen and gpt-oss samples, deduplicated, lexically guarded, REPL-checked in closeness order. This is the union-coverage interface: candidates from both families enter one pool; the compiler selects.
- **S2 — targeted repair, capped.** $\leq 2-3$ error-informed rounds with the model that wrote the candidate, deterministic error->fix rewrites applied first (e.g. "unknown constant X, did you mean Y" substitution; `set_option` insertions for heartbeat/recursion errors).
- **S3 — plateau handoff.** When the error signature repeats or rounds stop progressing, the *other* model receives the candidate plus a distilled error history — not a critique dialogue, a fresh attempt with more information, exploiting the cutoff gap (qwen knows 2026 Mathlib idioms; gpt-oss doesn't) and the stuck-vs-thrash asymmetry.
- **S4 — decomposition, the long-window weapon.** A sketcher (qwen-think primary, gpt-oss alternating) writes an informal solution and a Lean skeleton of standalone, explicitly-typed helper lemmas with `sorry` bodies. The skeleton is validated, broken lines are masked rather than discarded (DSP+), and holes are filled breadth-first — every hole gets a cheap pass before any hole gets a deep dialogue — by tactic cascade, then qwen waves, then gpt-oss escalation, with per-hole repair dialogues seeded from the best failure. Proven lemmas persist in a per-problem pool and survive re-sketches and restarts; the same error class twice triggers re-decomposition rather than more grinding.
- **S5 — endgame.** Stop LLM work with margin, re-verify the winner, restore pristine statements and imports, enforce literal answers, return.

4.2 Time constants that respect the window

Two of the highest-value changes of the selection loop (§5) were embarrassingly simple. *Window-proportional timeouts*: a 960-second gp-oss timeout can never fit a 20-minute eval window, which silently disabled decomposition at short caps — all model/stage time constants now scale with the window (with floors). *Cycle caps that respect remaining time*: the outer loop's cycle cap was calibrated for slow mixed-model cycles; when a provider outage made cycles fast, it stranded three hours of an eight-hour window — past the cap the loop now keeps cycling while ≥ 45 minutes remain.

4.3 The comparator precheck — aligning search with the actual gate

The single most valuable component was found by chasing a silent failure class: **REPL-accepted proofs kept timing out the Comparator's cold build** — three times on one problem, then again on another — kernel cost that warm checking cannot see. The agent now verifies accepted winners against the *real* Comparator (a fresh scoring container) before returning, when enough window remains. A precheck timeout demotes the proof to a stash-don't-discard fallback and the hunt continues for a lighter proof.

This closed the loop between search and scoring, and it was validated in both directions, repeatedly: one full-cap p09 run **ships 0/4 without it** (two would-be fatal proofs rejected before a light one passed); p10 converted after three straight timeout losses; and in the final validation runs the same save executed twice more — including once where the *fallback* half mattered: a precheck that timed out under machine load had rejected a perfectly fine proof, and only because rejected winners are stashed rather than discarded did it ship and pass (§6.4). When the scoring gate and the search-time checker disagree, only the gate is real.

5. Methodology

Matched-budget arms. Part Two compares *solo-qwen*, *solo-gptoss*, and *duo* — *the same ladder code*, with only the model set switched — at identical caps, so coordination is isolated from scaffolding. The kit's baselines additionally give a scaffold-free floor per model (§6.1).

A custom eval set, because iterating on 16 public problems overfits. We authored **24 new problems** in exact kit format (number theory, modular arithmetic, divisor counts, factorial valuations, inequalities, telescoping products), each admitted only after a reference proof was machine-verified through the REPL — an unprovable statement would silently poison every eval. Split: 16-problem dev set for selection, 8 held back for overfitting checks. The agent never sees the reference proofs.

Composition over totals. Repeated identical-config runs established a ± 2 noise band on the dev-16, driven by three coin-flip problems; three problems proved structurally unsolved at short windows. Variants were therefore judged on *which* problems they hold or flip, promotions required a repeat plus a held-set check, and totals inside the band were treated as noise.

Selection at scale. ~30 scored evaluations total: a seven-iteration selection loop, then a **ten-branch falsification sweep** — ten mechanisms from the literature and our own failure analysis (premise-name verification against the REPL, plan-then-sample, temperature-spread waves, error-clustered repair transplants, goal-typed fill prompts, [apply?](#) suggestion harvesting, strengthen-the-induction re-sketching, a cross-cycle critic, a two-skeleton portfolio, deterministic bounded-goal templates), each built on an independent branch behind a default-off flag and evaluated single-flag — then confirmation seeds, long-horizon transfer runs, and three full-cap validations at exact judge settings.

6. Results

6.1 Scaffold vs. raw model — the harness effect per model

The cleanest way to see what the coordination layer adds is to hold the model fixed and swap the scaffold. "Raw" is the kit's committed single-model repair-loop baseline (~20-minute effective caps); our arms ran the full ladder at 30-minute caps (two seeds) — caps differ slightly, so treat the raw column as a floor, not an exact control.

configuration	solved / 16	spend	notes
raw qwen (kit baseline)	7	—	thrash loops; missed a free <code>norm_num</code> problem in 25 paid turns
raw gpt-oss (kit baseline)	8	—	stuck loops; 18-min effective cap
union of raw baselines	10	—	the portfolio floor
our ladder, solo-qwen	8 / 9	\$0.45 / \$0.40	seed 2 solved hard-tier p10 at a 30-min cap
our ladder, solo-gptoss	8 / 9	\$0.04 / \$0.01	cheapest arm by ~10×
our ladder, duo	9 / 10	\$0.16 / \$0.17	equals the union of its own solo arms, at 1/3–1/2 solo-qwen's cost

Three readings. First, the scaffold alone is worth ~+1–2 problems to *each* model at short caps (and the free deterministic sweep covers 6/16 at \$0.00 before any model is called). Second, the duo beats the best solo arm by +1 on **both** seeds while being far cheaper than solo-qwen — the cascade only spends gpt-oss where qwen and free tactics failed. Third, the duo in a *single run* matches the union of the two solo arms: portfolio coverage is real, and the solos fail *different* problems.

But the short-cap table understates the scaffold, because the raw baselines' real ceiling was capability: nothing raw ever touched the hard tier.

6.2 The hard tier at exact judge settings (8 h / \$1.00)

Six sample problems were unsolved by both baselines. Our reference-proof work showed only four are actually provable — **one is mathematically false as formalized and one cannot elaborate under the Comparator's true imports** (undetectable via the kit's import-stripping REPL; documented with certificates in [RESEARCH.md](#)), so the scorable ceiling is 14/16. Against the four provable hard problems, across three full-cap validation runs plus long-horizon experiments:

problem	best outcome	evidence
p09 (IMO 1964)	Comparator PASS, 3-for-3 at full caps	\$0.10–0.91, ~2–7.6 h; every pass required the precheck to reject kernel-heavy proofs first
p10 (factorial powers)	Comparator PASS	first-ever hard solve at 2 h caps (\$0.05); converted at judge-compatible checking (\$0.029, 38 min) after the precheck landed
rmo_2000_2 (cube sandwich)	Comparator PASS	\$0.75, 7.4 h of accumulated lemma-building in one uninterrupted window
rmo_2001_2 (divisor pairs)	not solved	best attempt ended one unfilled hole from a complete skeleton

Cumulative demonstrated coverage: **13 of the 14 scorable sample problems**. The raw baselines' hard-tier score was 0.

6.3 The custom set: selection, plateau, and a first-ever solve

On the dev-16 the selection loop lifted the promoted configuration **from 11/16 to 13/16** (held set stable at 6/8 on both checks). The ten-branch sweep then failed to beat 13 — instructively. The best first seed (REPL-verified premise hints, 13/16 with every coin-flip problem held) collapsed to 10/16 on its confirmation seed: seed luck, not mechanism. Two mechanisms never fired (their trigger conditions are rare at 20-minute windows). And every mechanism that *added prompt material* —

plans, technique blocks, harvested suggestions — scored at or below defaults while costing more. At short windows, cycle count and sample diversity beat guidance; the tuned defaults are a genuine local optimum.

Where is the remaining headroom, then? We measured it: doubling the short window (20 to 40 min) moved *nothing* — the curve is flat where iteration is the input. But at **2-hour caps, h04 — one of three dev problems that had survived all ~19 short-window evaluations — fell** (comparator 43 s, \$0.16). Structure yields to time, not to prompting. This mirrors the kit hard tier exactly and is the empirical basis for the capability×time claim in §7.

6.4 Reliability results (they decided real points)

Week-long totals under adversarial conditions — container restarts chopping every in-flight connection, a ~14-hour provider outage on the gpt-oss channel, REPL container deaths under memory contention: zero unaccounted-cost failures in the final configuration; every full-cap point traces causally through the rails. The precheck alone flipped one 0/4 run to 1/4, converted p10's chronic loss, and (via its stash-don't-discard fallback) rescued the h04 first-ever from its own false-negative under machine load.

7. Part Two: does the collaboration beat either model alone — and why?

At matched budgets, yes, modestly and consistently; and the "why" decomposes. The duo was never worse than the best solo arm, beat it by one problem on both seeds, matched the union of the solos in one run, and cost a fraction of the stronger solo. Attribution, from ablations, per-problem event logs, and one natural experiment:

- 1. The verifier-centric scaffold is the largest term** — worth more than any model-pairing effect. It gives each model +1–2 problems at short caps, the entire hard tier at long caps, and its most valuable single component (the comparator precheck) is pure verifier alignment, no second model involved. The honest experiment that proves this: given 4 hours and the same scaffold, **solo-qwen also converted p09** — heavy proof precheck-rejected, lighter fill passed — so at long horizons the scaffold, not the pairing, carries p09.
- 2. Union coverage is the reliable collaboration gain.** Different families fail different problems (observed at every scale: baselines 7/8->10; our solo arms miss disjoint problems the duo holds). It requires no dialogue — one candidate pool, a compiler selecting.
- 3. Skeleton diversity occasionally matters decisively.** The first full-cap p09 pass grew from a **gpt-oss** sketch that qwen then repaired and filled — a proof the solo-qwen arm could not have produced. This is "one model filling the other's gaps" at its most concrete: different priors propose different proof *structures*, and the compiler doesn't care whose structure wins.
- 4. The answer-consensus gate is cheap insurance** on exactly the decision the verifier can't check; cross-model disagreement caught wrong answers before proof spend in live runs.
- 5. Robustness is a collaboration benefit we measured by accident.** During the final validations the gpt-oss provider refused requests for ~14 straight hours. A solo-gptoss submission would have scored zero that night. The duo degraded gracefully into solo-qwen behavior and kept solving — the portfolio argument in its bluntest form.
- 6. **What does *not* help, measured:**** conversation-style coordination (never built, on strong negative evidence we then corroborated: ten mechanism variants, several of them guidance-flavored, all at-or-below tuned defaults); deeper per-hole reasoning at short windows (+16% cost, no structural gain); gpt-oss inside short-window sampling waves (its latency halves qwen's cycle count).

The scientific summary: with a perfect verifier, collaboration pays through *coverage* (candidate and skeleton diversity across model families), through *one compact consensus decision* (answer commitment), and through *portfolio robustness* — not through models talking to each other. Depth on hard problems comes from capability×time under a decomposition scaffold: p09 needed ~2 hours, rmo_2000_2 needed 7.4, h04 needed 2, and no prompting trick at 20 minutes substituted for any of them. The one still-unsolved provable problem (rmo_2001_2, one hole short after ~4 h) is the same claim seen from below: the pair's joint capability at this time budget, not the coordination between them, is the binding constraint.

8. Limitations

Sample sizes are small (16 kit + 24 custom problems); we report exact per-problem outcomes and treat ±2 as noise rather than claiming significance. Dev-container weather (provider outages, container restarts, shared-box REPL contention) cost

several runs and is documented per-incident; judging hardware will be kinder, and the one dev-only mitigation (`VM_TRANSPORT_FAILURE_POLICY`) is never set at judging — default behavior is the kit's fail-closed semantics. The long-horizon solo comparison ran at 4 h vs. the duo's 8 h full-caps — a floor for the solo arm, not a matched control. And these are small models: the demonstrated band is "all of the easy/medium tier plus some genuinely hard problems, with IMO-tier still out of reach" — consistent with published results for similarly-sized backbones.

9. Conclusion

The coordination layer is unglamorous by design: let a compiler judge, buy coverage from two model families, spend consensus only on compact decisions, escalate depth — not chatter — on hard problems, and align the search loop with the exact gate that scores it. Under judge settings it converted a 0-for-hard-tier baseline pair into 3 of 4 provable hard problems solved, holds 13/14 scorable sample problems in cumulative coverage, and survived provider outages and machine failures with its accounting intact — for about \$16 of the \$50 key, spent mostly on evidence rather than solving.

Reproduction: `RUNBOOK.md` (one command per experiment; `scripts/judge_check.sh` passes on the submitted configuration). Every claim above traces to a run directory under `outputs/` and a row in `EXPERIMENTS.md` or `RESEARCH_LOOP.md`.